

# Neural Networks Essentials

## Semantic Segmentation

Denis C. Ilie-Ablachim

April 1, 2026

# Where we are

From Image Classification to Dense Prediction

What Makes Segmentation Hard?

From Sliding Windows to Modern Architectures

Data, Annotation, Losses, and Metrics

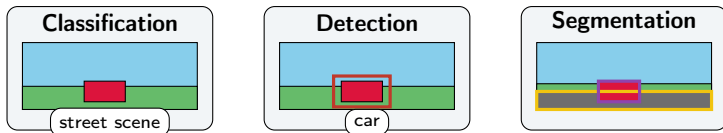
Practical Project Recipe

Case Studies and Discussion

Conclusion

# Where does semantic segmentation fit?

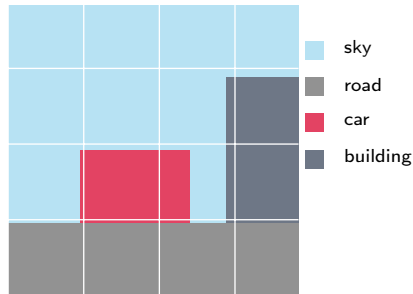
- **Image classification:** one label for the whole image.
- **Object detection (YOLO):** *where* objects are, using bounding boxes.
- **Semantic segmentation:** a class label for *every pixel*.



**Takeaway:** Segmentation turns vision from “what is in the image?” into “what is each part of the image?”

# What exactly is semantic segmentation?

- Input: an image of size  $H \times W \times 3$ .
- Output: a mask of size  $H \times W$ , where each pixel stores a class id.
- Example classes: road, building, car, person, sky.
- The network predicts **dense** outputs: one decision per pixel.



## Important distinction

Semantic segmentation groups all objects of the same class together. If two cars appear, both are labeled car; they are *not* separated as individual instances.

**Takeaway:** Think of segmentation as image classification repeated at every pixel, but with heavy sharing of computation and context.

# Semantic vs. instance vs. panoptic segmentation

| Task                         | What it predicts                        | Typical use case                                                 |
|------------------------------|-----------------------------------------|------------------------------------------------------------------|
| <b>Semantic segmentation</b> | Every pixel gets a class label          | Road scene understanding, land cover, organ regions              |
| <b>Instance segmentation</b> | Every object instance gets its own mask | Counting people, tracking products, separating overlapping cells |
| <b>Panoptic segmentation</b> | Combines both “stuff” and “things”      | Full-scene reasoning in robotics/autonomous systems              |

## Semantic

Two cars → same class color.

## Instance

Two cars → two separate masks.

## Panoptic

Road/sky/buildings + separate cars/people.

**Practical note:** If your audience already knows YOLO, instance segmentation is the closest mental bridge. Semantic segmentation is often simpler conceptually, but harder when boundaries matter.

# Why pixel-level prediction is useful

- **Autonomous driving:** drivable area, sidewalks, lanes, pedestrians.
- **Medical imaging:** tumor boundaries, organs, lesions.
- **Aerial/satellite:** water, roads, vegetation, buildings.
- **Manufacturing:** scratches, corrosion, missing material, solder defects.

## Why not just use boxes?

Bounding boxes are coarse. A box around a road crack, tumor, or lane marking is usually much less useful than the exact region.

## Typical business value

Segmentation often enables measurement: area, length, coverage, occupancy, distance to boundary, or change over time.

# Example-driven intuition

## Road scenes

You want to know where the road *is*, not just whether a road exists.

## Medical scans

The output is often used directly by clinicians or by a downstream measurement system.

## Agriculture / Earth observation

Area and shape of regions matter: crop stress, flooded regions, deforestation.

## Industrial inspection

Defects are often thin, irregular, and small. Boxes waste space and hide precision errors.

**Takeaway:** Segmentation is often chosen when **shape**, **extent**, or **precise location** matter.

# Where we are

From Image Classification to Dense Prediction

What Makes Segmentation Hard?

From Sliding Windows to Modern Architectures

Data, Annotation, Losses, and Metrics

Practical Project Recipe

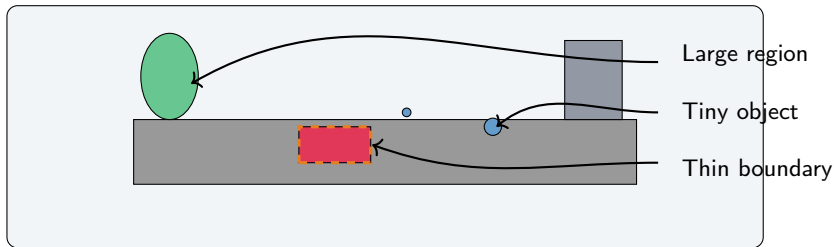
Case Studies and Discussion

Conclusion



# Why segmentation is harder than it looks

- Objects appear at many scales.
- Boundaries can be fuzzy or ambiguous.
- Small classes get overwhelmed by background.
- Lighting, weather, sensors, and viewpoints change.
- Labels can be noisy or inconsistent.



**Takeaway:** The model must combine **global context** (what scene am I in?) with **local detail** (where exactly is the boundary?).

# A useful mental model: three competing needs

## 1. Semantics

Recognize *what* something is.

## 2. Localization

Know *where* it is in the image.

## 3. Resolution

Preserve sharp *boundaries*.

## Why plain CNN classifiers are not enough

As feature maps become smaller and more abstract, the network gets semantics but loses exact spatial detail.

**Intuition:** Most segmentation architectures can be understood as ways to get all three at once.

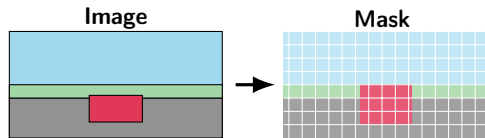
# What does the data look like?

## Training pair

- RGB image
- Segmentation mask with one integer per pixel

## Mask representation

- 0 = background
- 1 = road
- 2 = car
- 3 = pedestrian
- etc.



**Practical note:** A big source of mistakes is forgetting that every geometric transform applied to the image must also be applied to the mask.

# Where we are

From Image Classification to Dense Prediction

What Makes Segmentation Hard?

From Sliding Windows to Modern Architectures

Data, Annotation, Losses, and Metrics

Practical Project Recipe

Case Studies and Discussion

Conclusion

# Naive idea: classify every patch

- Slide a small window across the image.
- Feed each patch into a normal CNN classifier.
- Assign the center pixel (or the whole patch) to a class.

## Why it is attractive

You can reuse a classification network almost directly.

But...

Computationally wasteful and poor at using context.

**Takeaway:** Patch-based segmentation is conceptually simple, but computationally wasteful and poor at using context.

# Why the patch-based approach fails in practice

## Problem 1: redundant computation

Overlapping patches repeat almost the same convolutions thousands of times.

## Problem 2: weak global context

A small patch may not tell you whether a gray region is a road, wall, or roof.

**Intuition:** The same image structures are reused across neighboring pixels, so the model should share computation across the whole image.

## Problem 3: blocky outputs

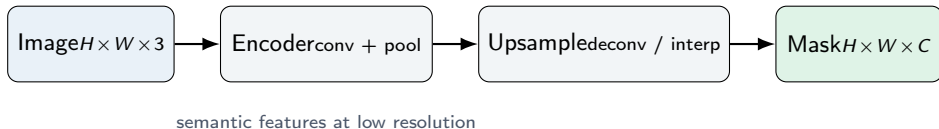
Predictions can be inconsistent from patch to patch, producing jagged masks.

## Problem 4: slow inference

Fine strides improve quality but make runtime explode.

# Key step forward: Fully Convolutional Networks (FCNs)

- Remove dense layers and keep only convolutional operations.
- Feed the whole image through the network in one pass.
- Predict a dense feature map, then upsample to input resolution.



## Big idea

A classifier can be turned into a dense predictor by keeping spatial structure all the way through the network.

# Encoder-decoder intuition

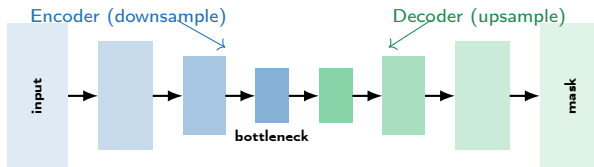
- **Encoder:** repeatedly downsamples the image and builds more abstract features.
- **Decoder:** gradually upsamples those features back to pixel resolution.

## What the encoder gives you

Context, semantics, robustness to local noise.

## What the decoder tries to recover

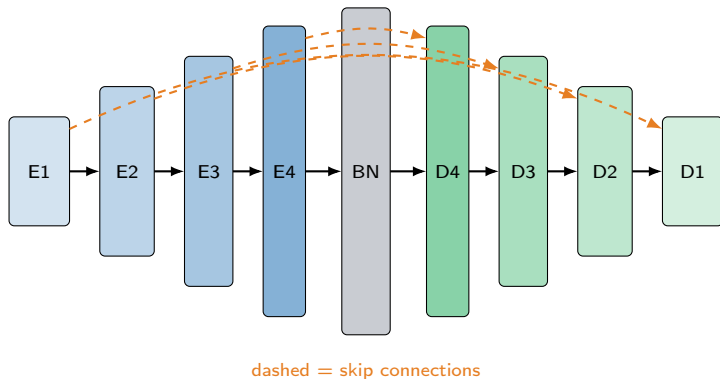
Edges, details, and exact object extent.





# U-Net: why skip connections are so useful

- U-Net sends high-resolution features from the encoder directly to matching decoder stages.
- This helps recover boundaries and small structures.

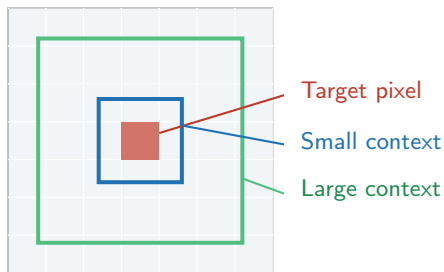


## Intuition

The encoder knows *what*; the early layers remember *where*. Skip connections bring back the “where.”

# Why context matters: receptive field and multi-scale reasoning

- A pixel often cannot be classified from its immediate neighborhood alone.
- The model needs a large **receptive field**: a wide enough portion of the image.
- Modern models use pyramids, dilation/atrous convolutions, or transformers to mix information across scales.



**Takeaway:** Good segmentation models are not just about sharper upsampling; they are also about seeing enough context to disambiguate the scene.

# Three architecture families you should recognize

| Family                       | Strengths                                                                                | Good when                                         |
|------------------------------|------------------------------------------------------------------------------------------|---------------------------------------------------|
| <b>FCN / encoder-decoder</b> | Simple concept, strong baseline                                                          | You want a clean starting point                   |
| <b>U-Net and variants</b>    | Great boundaries, excellent for limited data, widely used in medical/industrial settings | Precise localization matters                      |
| <b>DeepLab-style models</b>  | Strong context modeling and robust performance on natural scenes                         | Outdoor scenes, larger datasets, harder semantics |

## Rule of thumb

If you need a first practical baseline, start with a pretrained U-Net-like model or a lightweight DeepLab variant.

# What about YOLO-style segmentation?

- If you already use YOLO, segmentation extensions are a natural next step.
- These models usually add a **mask head** on top of the detector.
- Very practical when you need real-time speed and instance masks.

## Caution

YOLO segmentation is not always the best tool for broad “stuff” classes like sky, road, or tissue regions. Dedicated semantic segmentation models often produce cleaner full-scene masks.

## Best fit

Scenes with countable objects where each instance matters: products, people, vehicles, defects.

**Practical note:** Use YOLO-style segmentation when your team already has a detection workflow and cares about speed and instance-level outputs.

# Where we are

From Image Classification to Dense Prediction

What Makes Segmentation Hard?

From Sliding Windows to Modern Architectures

**Data, Annotation, Losses, and Metrics**

Practical Project Recipe

Case Studies and Discussion

Conclusion

# The hidden cost: annotation

- Segmentation labels are expensive because humans must draw regions, not just boxes.
- Boundaries can be subjective.
- Thin objects and tiny defects are especially time-consuming.

## Very common failure mode

The model is “wrong” because the labels are inconsistent, not because the architecture is weak.

### Practical annotation advice

1. Start with a clear class taxonomy.
2. Write labeling rules with visual examples.
3. Do a small pilot before full annotation.
4. Measure inter-annotator agreement on ambiguous classes.

# Dataset design: what to collect and how to split

- Cover important environments, sensors, seasons, lighting, and failure cases.
- Make sure rare classes are represented.
- Split by scene, patient, site, machine, or date when leakage is possible.

## Think about deployment first

Your validation set should look like the real operating conditions, not just random samples from the same recording session.

| Bad split                                   | Consequence                     |
|---------------------------------------------|---------------------------------|
| Random frames from same video in train/test | Over-optimistic score           |
| Same patient in train/test                  | Unrealistic medical performance |
| Same production batch in train/test         | Hidden domain leakage           |

# Data augmentation for segmentation

- Spatial: flip, rotate, crop, resize, elastic deformation.
- Appearance: brightness, contrast, blur, noise, color shifts.
- Domain-specific: weather simulation, motion blur, stain variation, sensor noise.

## Why augmentation helps

It teaches invariances and makes overfitting harder.

## Critical implementation detail

Spatial transforms must be applied identically to image and mask.

## Mask-specific detail

When resizing masks, use nearest-neighbor interpolation; otherwise class ids get corrupted.



# Loss functions: what are we optimizing?

- **Pixel-wise cross-entropy:** default choice for multi-class segmentation.
- **Dice loss:** focuses on region overlap; popular in medical imaging.
- **Focal loss:** useful when easy/background pixels dominate.
- **Hybrid losses:** often cross-entropy + Dice or boundary-aware terms.

## Intuition

Cross-entropy treats each pixel as a classification task.

Dice/IoU-style losses care more about the overlap of whole regions.

**Takeaway:** Choose the loss to match the pain point: class imbalance, overlap quality, or boundary precision.

# The class imbalance problem

- In many datasets, most pixels are easy background.
- The rare classes are often the ones you care about most.

## Examples

- Small pedestrians in a road scene
- Tiny tumor regions in a medical scan
- Micro-defects on a large metal surface

## What can go wrong?

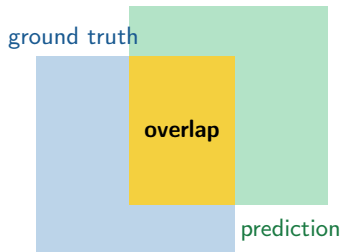
A model that predicts mostly background may achieve high pixel accuracy and still be useless.

## Countermeasures

Class weighting, focal loss, balanced crops, oversampling, and reporting per-class metrics.

# Metrics: why pixel accuracy is not enough

- **Pixel accuracy:** fraction of correctly labeled pixels.
- **IoU (Intersection over Union):**  $\text{IoU} = \frac{\text{overlap}}{\text{union}}$       **mIoU:** average IoU across classes.
- **Dice / F1:**  $\text{Dice} = \frac{2|A \cap B|}{|A| + |B|}$



$$\text{IoU} = \frac{\text{overlap}}{\text{GT} + \text{Pred} - \text{overlap}}$$

$$\text{Dice} = \frac{2 \times \text{overlap}}{\text{GT} + \text{Pred}}$$

## Rule of thumb

Always report per-class IoU or Dice, not only an overall score.

# Metric intuition with a simple example

Imagine an image where **95%** of pixels are background and **5%** belong to a crack.

If the model predicts *all background*:

- Pixel accuracy = 95%
- Crack IoU = 0
- Crack Dice = 0

**Takeaway:** A high accuracy score can hide a total failure on the class you care about.

## What to monitor in practice

- mIoU
- Per-class IoU
- Visual validation examples
- Calibration / confidence maps if available

# What good evaluation looks like

1. Report numbers **and** show masks side by side.
2. Include easy, medium, and failure-case examples.
3. Break results down by class, environment, device, or site.
4. Measure boundary quality if that matters operationally.
5. Test on realistic out-of-distribution conditions.

## For technical teams

A confusion matrix is still useful, but now it is a *pixel confusion matrix*: which classes are confused with which other classes across all pixels.

## Never ship from metrics alone

A segmentation model with decent mIoU can still produce unacceptable failure modes on rare but critical cases.

# Where we are

From Image Classification to Dense Prediction

What Makes Segmentation Hard?

From Sliding Windows to Modern Architectures

Data, Annotation, Losses, and Metrics

Practical Project Recipe

Case Studies and Discussion

Conclusion

# A minimal practical training recipe

```
# image: [B, 3, H, W]
# mask: [B, H, W] with integer class ids
logits = model(image) # [B, C, H, W]
loss_ce = F.cross_entropy(logits, mask)
loss_dice = dice_loss(logits, mask)
loss = loss_ce + 0.5 * loss_dice

optimizer.zero_grad()
loss.backward()
optimizer.step()
```

- Use a pretrained encoder if possible.
- Start simple before adding exotic tricks.
- Inspect predictions every epoch, not just scalar losses.

**Practical note:** Many failed projects could have been rescued earlier by visually checking masks during training.

# A practical checklist for your first project

## Start here

1. Define the classes and the operational goal.
2. Build a small, clean pilot dataset.
3. Train a strong baseline with a pretrained encoder.
4. Review predictions visually with domain experts.

## Then improve

1. Fix label inconsistencies.
2. Add hard examples and augmentation.
3. Tackle class imbalance.
4. Compare models only after the pipeline is stable.

**Takeaway:** Dataset quality and evaluation design usually matter more than switching from one famous architecture to another.



# Choosing a model in practice

| Situation                               | Good starting choice        | Why                                |
|-----------------------------------------|-----------------------------|------------------------------------|
| Limited data, precise boundaries        | U-Net variant               | Strong localization, easy to train |
| Street scenes, good GPU, natural images | DeepLab-style model         | Strong context handling            |
| Real-time object masks                  | YOLO-style segmentation     | Speed and detection ecosystem      |
| Tiny deployment target                  | Lightweight encoder-decoder | Lower memory / latency             |

## Simple strategy

Start with the lightest model that can meet the accuracy requirement. Then scale up only if needed.

# Common failure modes and how to debug them

| Symptom                           | Likely cause                           | First thing to try                            |
|-----------------------------------|----------------------------------------|-----------------------------------------------|
| Everything becomes background     | Class imbalance / weak labels          | Weighted loss, balanced crops, more positives |
| Jagged boundaries                 | Decoder too weak / resolution too low  | Skip connections, higher input resolution     |
| Confusion between similar classes | Not enough context or ambiguous labels | Larger receptive field, better label rules    |
| Good validation, poor deployment  | Data leakage or domain shift           | Fix split, collect realistic validation data  |

# Deployment trade-offs

- Higher resolution helps small objects but increases memory and latency.
- Larger models improve accuracy but may be too slow for edge devices.
- Post-processing can clean masks but adds complexity.

## Think in system terms

The model is only one part of the pipeline: capture, preprocessing, inference, post-processing, monitoring, and retraining.

## Questions to ask before deployment

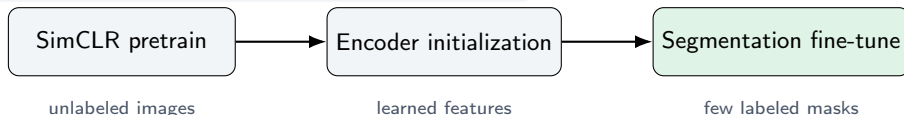
- Maximum latency?
- GPU or CPU only?
- Is calibration needed?
- What is the cost of false positives vs. false negatives?

# How SimCLR connects to segmentation

- SimCLR and related self-supervised methods learn good visual features without manual labels.
- Those features can initialize the encoder of a segmentation model.
- This is especially useful when mask annotation is expensive.

## Practical implication

If you have lots of unlabeled domain images and only a few segmentation masks, self-supervised pretraining can help.



# Where we are

From Image Classification to Dense Prediction

What Makes Segmentation Hard?

From Sliding Windows to Modern Architectures

Data, Annotation, Losses, and Metrics

Practical Project Recipe

Case Studies and Discussion

Conclusion

# Case study 1: road-scene segmentation

## Typical classes

Road, sidewalk, lane markings, vehicle, pedestrian, traffic sign, sky, building.

## What makes it hard

Night scenes, rain, reflections, motion blur, rare classes, thin structures.

## What matters operationally

- Drivable area quality
- Small pedestrian recall
- Robustness to weather and unseen cities
- Real-time latency

**Takeaway:** In road scenes, segmentation is part of a larger perception stack, so accuracy alone is not enough; robustness and latency matter just as much.

## Case study 2: medical / industrial segmentation

### Why U-Net became famous here

It works well when data is limited, boundaries matter, and local detail is critical.

### Typical pain points

- Tiny target regions
- Annotation variability
- Domain shift between devices/sites

**Practical note:** For many applied teams, the winning strategy is not “the fanciest model” but a clean annotation protocol plus a strong U-Net baseline.

### Useful evaluation extras

- Boundary metrics
- Area/volume error
- Object-level detection of small lesions/defects
- Human review of hard cases

# Discussion prompts for the audience

1. In your domain, do you care more about **area estimation**, **object counting**, or **boundary precision**?
2. Which classes would be expensive to miss? Which false positives are acceptable?
3. Is annotation cost the main bottleneck, or deployment latency, or domain shift?
4. Would detection, semantic segmentation, or instance segmentation be the right task formulation?

## Teaching goal

A large part of success in segmentation is choosing the right task and evaluation protocol before touching the model.



# Where we are

From Image Classification to Dense Prediction

What Makes Segmentation Hard?

From Sliding Windows to Modern Architectures

Data, Annotation, Losses, and Metrics

Practical Project Recipe

Case Studies and Discussion

Conclusion

# Key takeaways

1. Semantic segmentation predicts a class for **every pixel**.
2. The central challenge is balancing **semantics**, **context**, and **spatial detail**.
3. Encoder-decoder models, especially **U-Net**-style and **DeepLab**-style architectures, are the main practical tools.
4. In real projects, **labels**, **splits**, **augmentations**, and **metrics** matter as much as the model.
5. Always evaluate visually and per class; pixel accuracy alone can be misleading.

**Takeaway:** If YOLO taught us to draw boxes fast, semantic segmentation teaches us to reason about *surfaces, regions, and boundaries*.

**Classification:** what is in the image?

**Detection:** where are the objects?

**Semantic segmentation:** what is each pixel?

---

**Practical recipe:** clean labels + pretrained encoder + strong baseline + per-class metrics + visual inspection

# References and further resources

-  J. Long, E. Shelhamer, T. Darrell, *Fully Convolutional Networks for Semantic Segmentation*, CVPR 2015.
-  O. Ronneberger, P. Fischer, T. Brox, *U-Net: Convolutional Networks for Biomedical Image Segmentation*, MICCAI 2015.
-  L.-C. Chen et al., *Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation*, ECCV 2018.
-  M. Cordts et al., *The Cityscapes Dataset for Semantic Urban Scene Understanding*, CVPR 2016.
-  M. Everingham et al., *The PASCAL Visual Object Classes Challenge*, IJCV 2010.